

CODE ALPHA

Internship Report | Machine Learning Domain

INTERN PROFILE

Asadullah Khan

Machine Learning Intern

Email: ak5963905@gmail.com

Phone: 03132478703

Month 1 | 2025

Project

Credit Card Creditworthiness Classifier

submissions.codealpha@gmail.com | Code Alpha

1. Introduction

This report documents the work completed during Month 1 of the Machine Learning internship at Code Alpha. The internship focused on developing a practical, real-world machine learning solution for credit risk assessment using Python, pandas, scikit-learn, and XGBoost. The project involved end-to-end development of a Credit Card Creditworthiness Classifier — from raw data ingestion and cleaning to model training, evaluation, and comparison.

2. Intern Information

Full Name	Asadullah Khan
Internship Domain	Machine Learning
Email Address	ak5963905@gmail.com
Phone Number	03132478703
Organization	Code Alpha
Duration	Month 1
Submission Email	submissions.codealpha@gmail.com

3. Project — Credit Card Creditworthiness Classifier

3.1 Project Overview

The Credit Card Creditworthiness Classifier is a multi-class classification system designed to predict the credit score category of a customer based on their financial profile and payment history. It classifies customers into three categories: Poor, Standard, and Good credit. The project leverages a 50,000-record real-world dataset and applies a complete ML pipeline including data cleaning, feature engineering, encoding, outlier handling, and model comparison between Random Forest and XGBoost.

3.2 Dataset Description

The dataset contains 50,000 customer records with 27 raw features covering income, debt, payment behaviour, credit history, and more. After preprocessing, 20 meaningful features were retained for modelling.

Feature	Type	Description
Annual_Income	float64	Yearly income of the customer (USD)
Monthly_Inhand_Salary	float64	Net monthly take-home salary
Outstanding_Debt	float64	Remaining debt to be paid (USD)
Credit_Utilization_Ratio	float64	Balance / credit limit ratio (%)
Credit_History_Age	int64	Length of credit history (months)
Num_of_Delayed_Payment	int64	Avg. number of payments delayed
Delay_from_due_date	int64	Avg. days delayed from due date

Payment_of_Min_Amount	int (encoded)	Whether min amount paid (0/1)
Credit_Mix	int (encoded)	Credit portfolio quality (0/1/2)
Payment_Behaviour	OHE	Spending & payment pattern (6 cats)
Occupation	OHE	Customer job type (15 categories)
Total_EMI_per_month	float64	Monthly EMI obligations (USD)
Num_Credit_Inquiries	float64	Number of recent credit inquiries
Num_of_Loan	int64	Total number of active loans
Interest_Rate	int64	Interest rate on credit card (%)

3.3 Key Features of the Project

- Multi-class classification: Poor / Standard / Good credit score
- Complete ML pipeline from raw data to trained model
- Robust data cleaning: junk values, dirty numerics, complex string parsing
- Credit_History_Age parsed from "22 Years and 9 Months" format to total months
- Outlier handling via IQR Winsorization and business-logic hard caps
- Label Encoding for ordinal features, One-Hot Encoding for nominal features
- Model comparison: Random Forest vs XGBoost with detailed evaluation
- Confusion matrix analysis and per-class precision, recall, F1-score

3.4 Workflow

Step	Description
Step 1 — Data Loading & Exploration	Load 50,000 records, inspect dtypes, null counts, unique values
Step 2 — Feature Selection	Drop identifiers (ID, SSN, Name) and low-signal columns (Type_of_Loan)
Step 3 — Data Cleaning	Fix junk strings (____, !@9#%8), strip trailing underscores, parse formats
Step 4 — Type Conversion	Convert object columns to float/int; parse Credit_History_Age to months
Step 5 — Null Imputation	Mode for categoricals (Credit_Mix, Occupation); Median for numerics
Step 6 — Outlier Handling	IQR Winsorization for skewed cols; hard caps for business-logic cols
Step 7 — Encoding	Label encode ordinal cols; One-Hot encode nominal cols; bool → int
Step 8 — Model Training	Train Random Forest (100 trees) and XGBoost (200 estimators)
Step 9 — Evaluation & Comparison	Accuracy, Classification Report, Confusion Matrix for both models

3.5 Technologies & Libraries

Technology	Purpose
Python 3.x	Core programming language

Pandas	Data loading, cleaning, and manipulation
NumPy	Numerical operations and array handling
Scikit-learn	RandomForestClassifier, LabelEncoder, train_test_split, metrics
XGBoost	XGBClassifier — gradient boosting model
Matplotlib / Seaborn	Boxplots, histograms, feature importance visualization
Regex (re)	Parsing Credit_History_Age string format

4. Model Evaluation

4.1 Random Forest Results

Trained with 100 estimators, no max_depth restriction, random_state=42, using all CPU cores.

Class	Precision	Recall	F1-Score	Support
Poor (0)	0.80	0.94	0.86	1,296
Standard (1)	0.92	0.81	0.86	3,728
Good (2)	0.78	0.89	0.83	1,630
Accuracy			0.85	6,654
Macro Avg	0.83	0.88	0.85	6,654
Weighted Avg	0.86	0.85	0.85	6,654

Confusion Matrix — Random Forest

	Predicted			
	Poor	Std	Good	
Actual Poor	[1214	82	0]	<- Never confuses Poor as Good
Actual Std	[305	3021	402]	<- Main confusion zone (boundary class)
Actual Good	[0	183	1447]	<- Never confuses Good as Poor

4.2 XGBoost Results

Trained with 200 estimators, max_depth=6, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8.

Class	Precision	Recall	F1-Score	Support
Poor (0)	0.79	0.92	0.85	1,296
Standard (1)	0.92	0.80	0.86	3,728
Good (2)	0.78	0.90	0.83	1,630
Accuracy			0.85	6,654
Macro Avg	0.83	0.87	0.85	6,654

Weighted Avg	0.86	0.85	0.85	6,654
--------------	------	------	------	-------

4.3 Model Comparison

Aspect	Random Forest	XGBoost
Accuracy	85.39% ✓ Winner	84.79%
Training Style	Bagging (parallel trees)	Boosting (sequential trees)
Overfitting Risk	Lower	Slightly higher without tuning
Speed	Fast (n_jobs=-1)	Fast (n_jobs=-1)
Feature Importance	Built-in	Built-in
Tuning Needed	Minimal	Benefits from more estimators
Best For	Baseline & production	Competitions & fine-tuning

5. Implementation Code

5.1 Data Cleaning & Type Conversion

```
import pandas as pd, numpy as np, re

# Drop identifier & low-signal columns
data = data.drop(['ID', 'Customer_ID', 'Month', 'Name', 'Age', 'SSN', 'Type_of_Loan'], axis=1)

# Replace junk string values with NaN
data['Occupation'] = data['Occupation'].replace('_____', np.nan)
data['Credit_Mix'] = data['Credit_Mix'].replace('_', np.nan)
data['Payment_Behaviour'] = data['Payment_Behaviour'].replace('!@9#%8', np.nan)
data['Payment_of_Min_Amount'] = data['Payment_of_Min_Amount'].replace('NM', np.nan)

# Strip non-numeric chars & convert to float
dirty_cols = ['Annual_Income', 'Num_of_Loan', 'Num_of_Delayed_Payment',
              'Outstanding_Debt', 'Changed_Credit_Limit',
              'Amount_invested_monthly', 'Monthly_Balance']

for col in dirty_cols:
    data[col] = pd.to_numeric(data[col].astype(str).str.replace(r'^0-9.-', '', regex=True), errors='coerce')

# Parse Credit_History_Age: "22 Years and 9 Months" -> 273 months
def parse_credit_age(val):
    if pd.isna(val): return np.nan
    y = re.search(r'(\d+)\s*Year', str(val))
    m = re.search(r'(\d+)\s*Month', str(val))
    return (int(y.group(1))*12 if y else 0) + (int(m.group(1)) if m else 0)
data['Credit_History_Age'] = data['Credit_History_Age'].apply(parse_credit_age)
```

5.2 Null Imputation & Outlier Handling

```
# Impute nulls
cat_null_cols = ['Credit_Mix', 'Payment_of_Min_Amount', 'Payment_Behaviour', 'Occupation']
for col in cat_null_cols:
    data[col] = data[col].fillna(data[col].mode()[0])

data['Changed_Credit_Limit'] = data['Changed_Credit_Limit'].fillna(data['Changed_Credit_Limit'].median())

# IQR Winsorization
winsorize_cols = ['Annual_Income', 'Monthly_Inhand_Salary', 'Num_Bank_Accounts',
                  'Num_Credit_Card', 'Num_of_Delayed_Payment', 'Num_Credit_Inquiries',
                  'Outstanding_Debt', 'Total_EMI_per_month', 'Amount_invested_monthly', 'Monthly_Balance']
for col in winsorize_cols:
```

```

Q1, Q3 = data[col].quantile(0.25), data[col].quantile(0.75)
IQR = Q3 - Q1
data[col] = data[col].clip(Q1 - 1.5*IQR, Q3 + 1.5*IQR)

# Hard caps for business-logic columns
data['Interest_Rate'] = data['Interest_Rate'].clip(1, 36)
data['Delay_from_due_date'] = data['Delay_from_due_date'].clip(0, 60)
data['Credit_Utilization_Ratio'] = data['Credit_Utilization_Ratio'].clip(0, 100)

```

5.3 Encoding

```

# Label Encode ordinal features
data['Credit_Mix'] = data['Credit_Mix'].map({'Bad':0, 'Standard':1, 'Good':2})
data['Payment_of_Min_Amount'] = data['Payment_of_Min_Amount'].map({'No':0, 'Yes':1})

# One-Hot Encode nominal features
data = pd.get_dummies(data, columns=['Occupation', 'Payment_Behaviour'], drop_first=False)

# Convert bool -> int (pandas 2.0+ returns bool from get_dummies)
bool_cols = data.select_dtypes(include="bool").columns
data[bool_cols] = data[bool_cols].astype(int)

```

5.4 Model Training & Evaluation

```

from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Train/Test Split
X = data.drop('Credit_Score', axis=1)
y = data['Credit_Score']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
print(f'RF Accuracy: {accuracy_score(y_test, y_pred_rf)*100:.2f}%')

# XGBoost
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)

```

```

y_test_enc = le.transform(y_test)
cat_cols = X_train.select_dtypes(include="category").columns
X_train[cat_cols] = X_train[cat_cols].astype(int)
X_test[cat_cols] = X_test[cat_cols].astype(int)

xgb = XGBClassifier(n_estimators=200, max_depth=6, learning_rate=0.1,
                    subsample=0.8, colsample_bytree=0.8,
                    random_state=42, n_jobs=-1, eval_metric="mlogloss")
xgb.fit(X_train, y_train_enc, eval_set=[(X_test, y_test_enc)], verbose=50)
y_pred_xgb = xgb.predict(X_test)
print(f'XGB Accuracy: {accuracy_score(y_test_enc, y_pred_xgb)*100:.2f}%')
print(classification_report(y_test_enc, y_pred_xgb, target_names=['Poor', 'Standard', 'Good']))

```

6. Key Learnings & Skills Acquired

- Hands-on experience with the complete ML pipeline from raw messy data to a trained production-ready model
- Identifying and handling real-world data quality issues: junk strings, corrupt values, wrong dtypes, and outliers
- Choosing the right encoding strategy: Label Encoding for ordinal vs One-Hot for nominal features
- Understanding IQR Winsorization vs hard-cap outlier strategies based on business domain knowledge
- Training and comparing ensemble models — Random Forest (bagging) vs XGBoost (boosting)
- Interpreting multi-class classification reports: precision, recall, F1-score per class
- Analysing confusion matrices to understand boundary-class misclassification patterns
- Writing clean, modular, and well-commented Python code for ML projects

7. Conclusion

During Month 1 of the Machine Learning internship at Code Alpha, a fully functional Credit Card Creditworthiness Classifier was successfully developed and documented. The project tackled a real-world dataset with significant data quality issues and required careful domain-driven decision-making at every preprocessing step.

Random Forest achieved 85.39% accuracy and marginally outperformed XGBoost (84.79%) on the current configuration. Both models demonstrated strong performance with no cross-class confusion between the extreme classes (Poor and Good), confirming the quality of the feature engineering pipeline.

The internship has provided invaluable hands-on experience bridging theoretical ML knowledge with practical real-world problem-solving, and has built a solid foundation for advanced model tuning, deployment, and production-grade ML systems.

I hereby declare that the project and work described in this report are my own original work, completed during the internship period. All code has been written, tested, and documented by me.

Signature: _____ Date: _____

Asadullah Khan | Machine Learning Intern | Code Alpha